

# lab\_05

September 30, 2026

## 1 Lab: Extreme Events

### 1.1 Modeling the Unusual

- For a given sample, the maximum and minimum are fixed
- But if we have a large number of samples, we can study the distribution/density of those max/min draws
- Imagine:
  - The biggest storm in a year
  - The worst loss for the market in a month
  - The biggest earthquake in a decade
  - The longest time between goals in a game
- This is how we understand the distribution/density of extreme events
- The ***k*-th order statistic** is the *k*-th largest value in a sample of *n* from some distribution
- The minimum is the first order statistic and the maximum is the *n*-th order statistic

### 1.2 Example: Drawing from a Gaussian

Below, we find the minimum of a sample of size *n* drawn from a normal distribution, and then repeat this *b* times, to get a distribution of the minimum with 10000 counts in it. This is the distribution of the 1st order statistic, shown below in blue.

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

rng = np.random.default_rng( seed = 100 ) # Create a random number stream

n = 7 # Sample Size
b = 10_000 # Number of Simulations

sample = rng.standard_normal( size = (n,b) ) # Draw a sample from the standard
↪normal distribution
sorted_sample = np.sort(sample, axis=0) # Sort each column (each simulated
↪sample of size n)

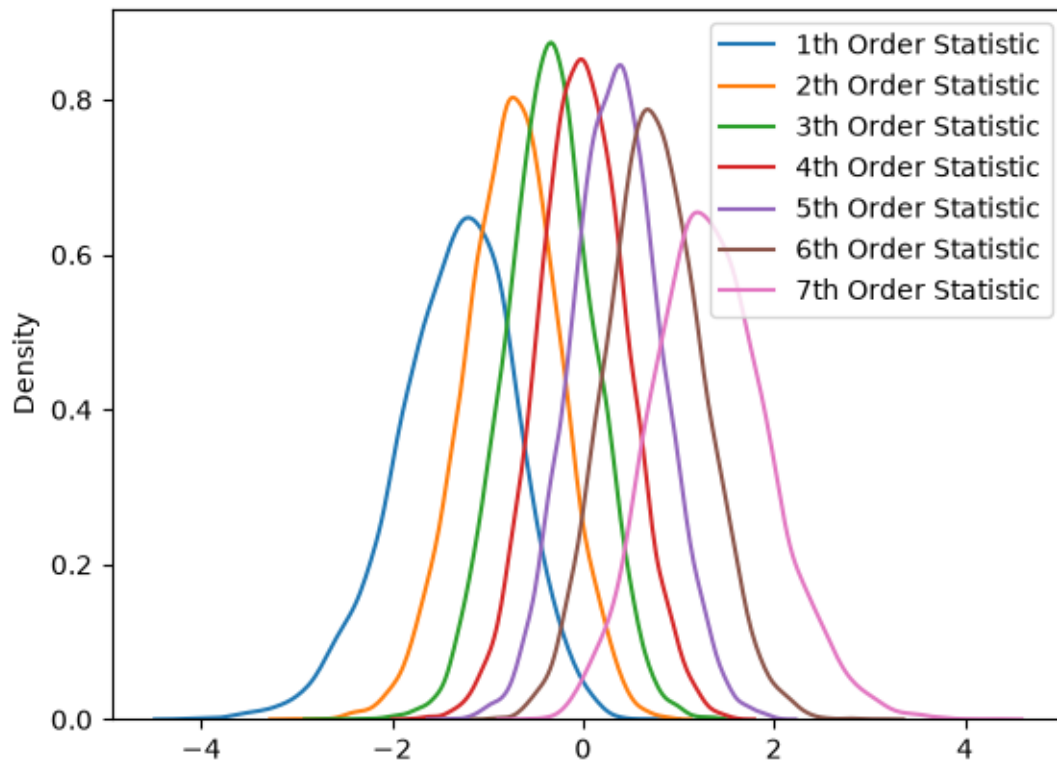
for j in range(n): # For each rank
```

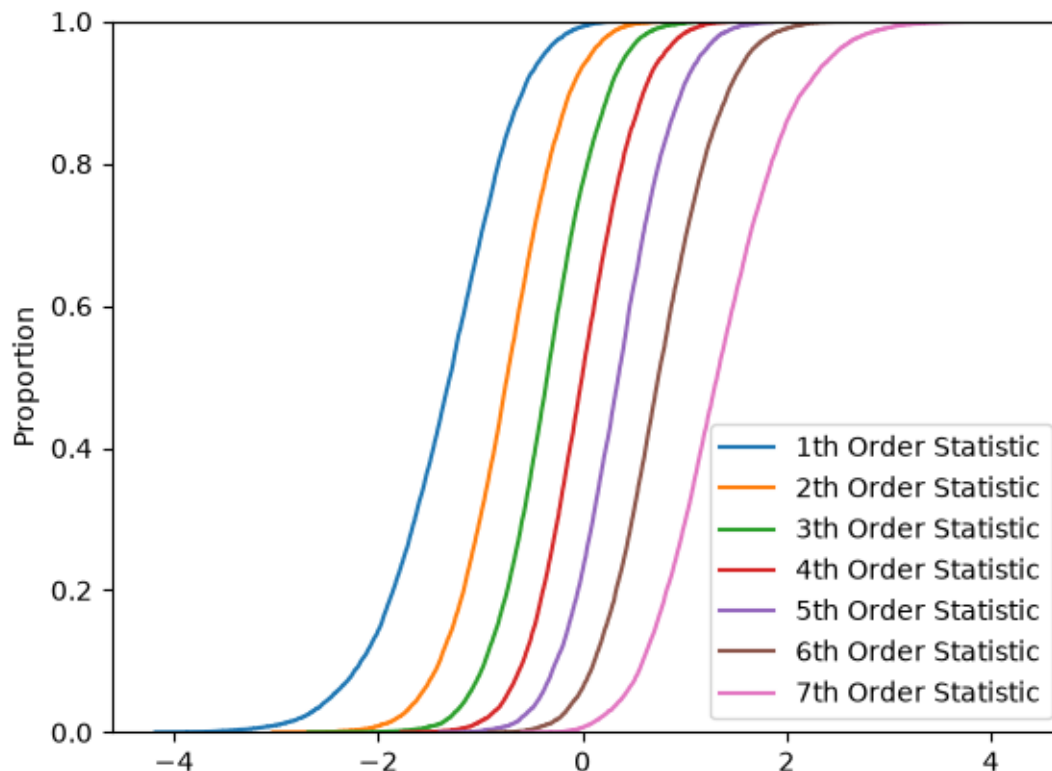
```

sns.kdeplot( sorted_sample[j,:], label=f'{1+j}th Order Statistic') # Plot the KDE
plt.legend() # Add legend
plt.show()

for j in range(n): # For each rank
    sns.ecdfplot( sorted_sample[j,:], label=f'{1+j}th Order Statistic') # Plot the ECDF
plt.legend() # Add legend
plt.show()

```





## 1.3 Analytic Relationship

### 1.3.1 Maximum

If  $F(x)$  is the underlying true CDF, let's consider the analytical equation for the distribution of the maximum of a sample of size  $n$ . (The maximum is its own random variable, with its distribution plotted above. What is this in math?) To build the CDF of the maximum, we ask (as always): "What's the probability the maximum lands at or below  $x$ ?" for every  $x$ . In other words:

$$F_{max}(x) = P(X_{max} \leq x)$$

This is nothing new, just applying it to the distribution of a specific order statistic, the max.

So what is the **probability that all of the draws are less than or equal to  $x$** , i.e. what is the probability that the maximum is less than or equal to  $x$  (the right hand side)? Because each draw is independent:

$$P(X_1 \leq x \wedge \dots \wedge X_n \leq x) = P(X_1 \leq x) \cdot P(X_2 \leq x) \cdot \dots \cdot P(X_n \leq x)$$

Because we are pulling from the same distribution every time, **each of these probabilities equals our CDF,  $F(x)$** :

$$P(X_1 \leq x, \dots, X_n \leq x) = F(x) \cdot F(x) \dots \cdot F(x)$$

The probability that all of the draws are less than or equal to  $x$  is therefore:

$$F_{\max}(x) = F(x) \cdot F(x) \cdots F(x) = F(x)^n$$

What is its density?

$$\begin{aligned} F'_{\max}(x) &= n * F_{\max}(x)^{n-1} * F'_{\max}(x) \\ F'_{\max}(x) &= n * F_{\max}(x)^{n-1} * f_{\max}(x) \end{aligned}$$

## 1.4 Minimum

The distribution of the minimum is somewhat harder, because our CDF is asking  $\leq x$ , and “max  $\leq x$ ” means “all  $\leq x$ ” but “min  $\leq x$ ” does not mean “all  $\leq x$ ”. We can’t just substitute in multiple copies of  $F(x)$ . Instead of looking at the probability that all draws are less than or equal to  $x$ , we need to know the probability that at least one is.

We want to ask “is the minimum less than or equal to  $x$ ?” which is the opposite of asking “are all of the draws, including the minimum, above  $x$ ?” The probability that all observations are above  $x$  is  $(1 - F(x))^n$ . Because this is the opposite of the question we actually want to ask, the complementary probability is what we truly want:

$$F_{\min}(x) = 1 - (1 - F(x))^n$$

What is its density?

$$\begin{aligned} F'_{\min}(x) &= -n(1 - F(x))^{n-1} * -F'(x) \\ F'_{\min}(x) &= n(1 - F(x))^{n-1} * f(x) \end{aligned}$$

## 1.5 All of the other order statistics, generally

- If you’re curious, the density of the  $k$ -th of  $n$  order statistics is

$$f_k(x) = \frac{n!}{(k-1)!(n-k)!} F(x)^{k-1} (1 - F(x))^{n-k} f(x)$$

- We’ll use the results for the minimum and maximum

## 1.6 The Data

We have three datasets:

1. Rainfall *totals* for every state-month from 1981 to 2025, with 5 reporting stations per state each month
2. Rainfall for every *station-day* over that same 1981-2025 period (same 5 stations per state) — much bigger, and used for a single question near the end of Step One
3. Daily and monthly U.S. stock market returns from 1926 to 2026

We want to understand tail behavior, to better reason about risk (engineering, financial, etc.).

```
[4]: df = pd.read_parquet("ghcn_prcp_monthly.parquet")
df.head()
```

```
[4]:   state state_name  station_rank  station_id station_name  latitude \
0    AK    Alaska           1  USW00027502   BARROW AP    71.2869
1    AK    Alaska           1  USW00027502   BARROW AP    71.2869
2    AK    Alaska           1  USW00027502   BARROW AP    71.2869
3    AK    Alaska           1  USW00027502   BARROW AP    71.2869
4    AK    Alaska           1  USW00027502   BARROW AP    71.2869

   longitude  elevation_m  year  month month_start  prcp_total_mm \
0  -156.7394           8.2  1981      1  1981-01-01         5.900001
1  -156.7394           8.2  1981      2  1981-02-01         1.600000
2  -156.7394           8.2  1981      3  1981-03-01         0.900000
3  -156.7394           8.2  1981      4  1981-04-01         5.100000
4  -156.7394           8.2  1981      5  1981-05-01         1.700000

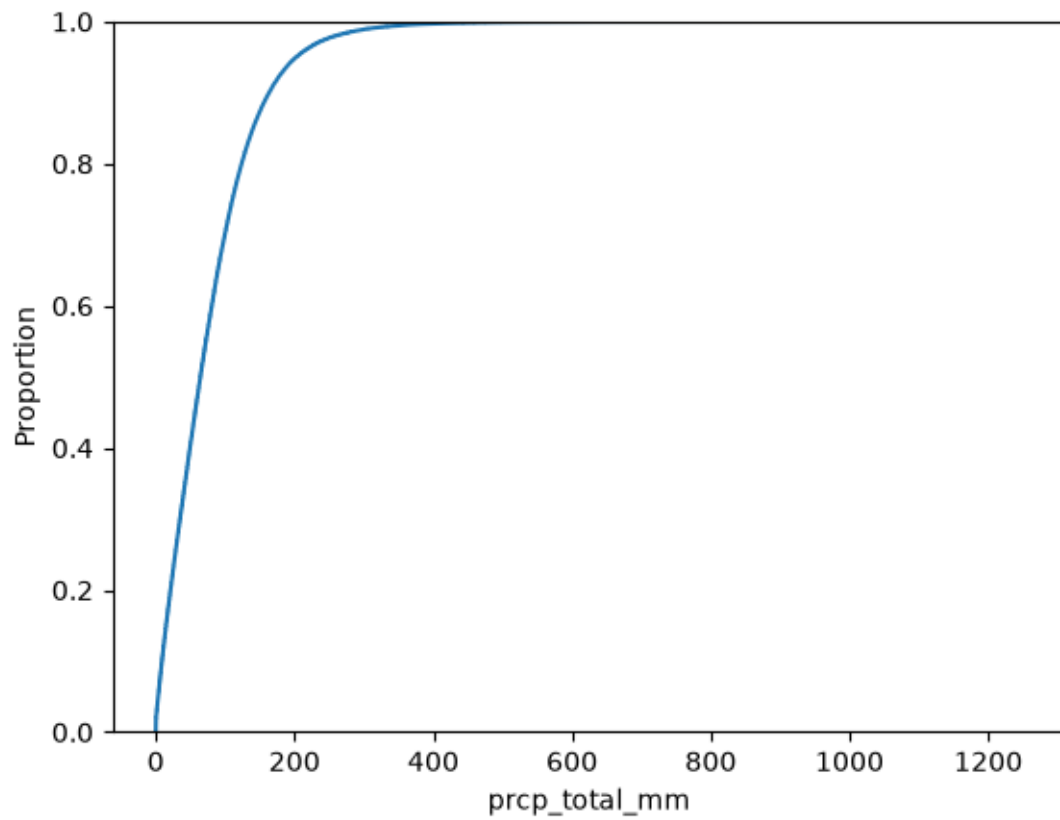
   observed_total_mm  observed_days  days_in_month  coverage_fraction \
0           5.900001             31             31                1.0
1           1.600000             28             28                1.0
2           0.900000             31             31                1.0
3           5.100000             30             30                1.0
4           1.700000             31             31                1.0

   prorated  imputed
0     False     False
1     False     False
2     False     False
3     False     False
4     False     False
```

**Step One.** 1. Plot the ECDF and KDE of `prcp_total_mm` with seaborn— the distribution and density of rainfall across the entire U.S.

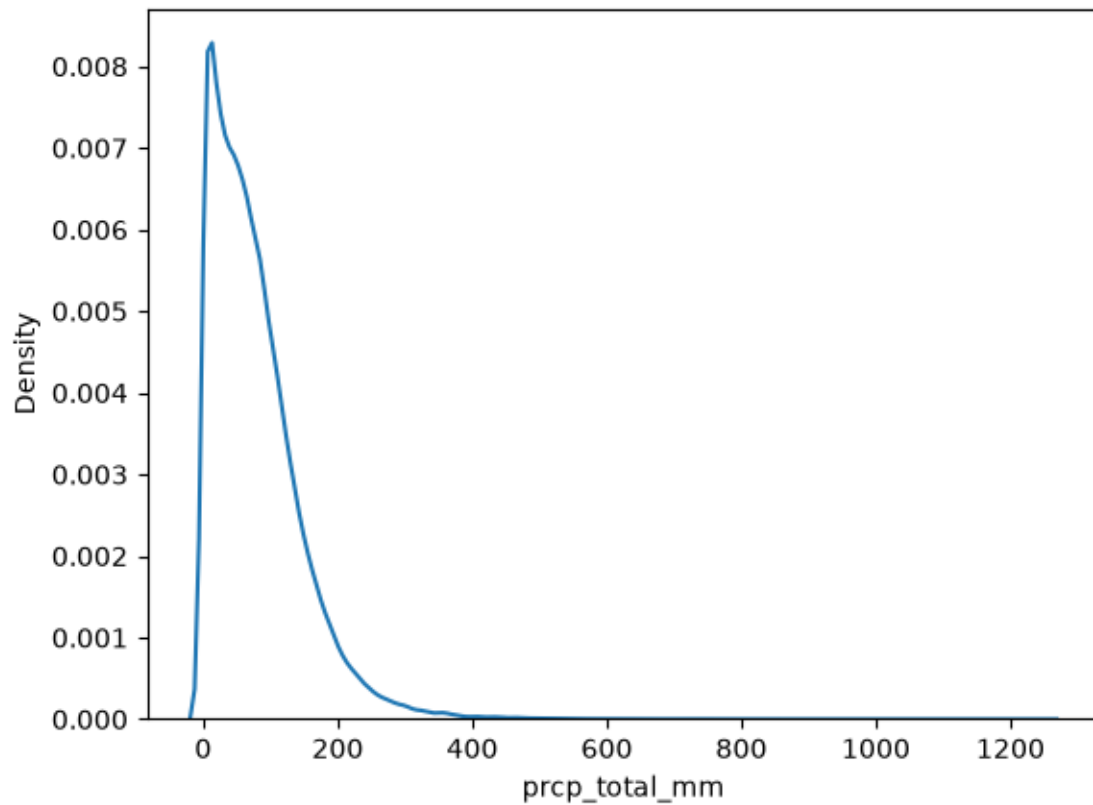
```
[5]: sns.ecdfplot( data = df, x = 'prcp_total_mm')
```

```
[5]: <Axes: xlabel='prcp_total_mm', ylabel='Proportion'>
```



```
[6]: sns.kdeplot( data = df, x = 'prcp_total_mm')
```

```
[6]: <Axes: xlabel='prcp_total_mm', ylabel='Density'>
```

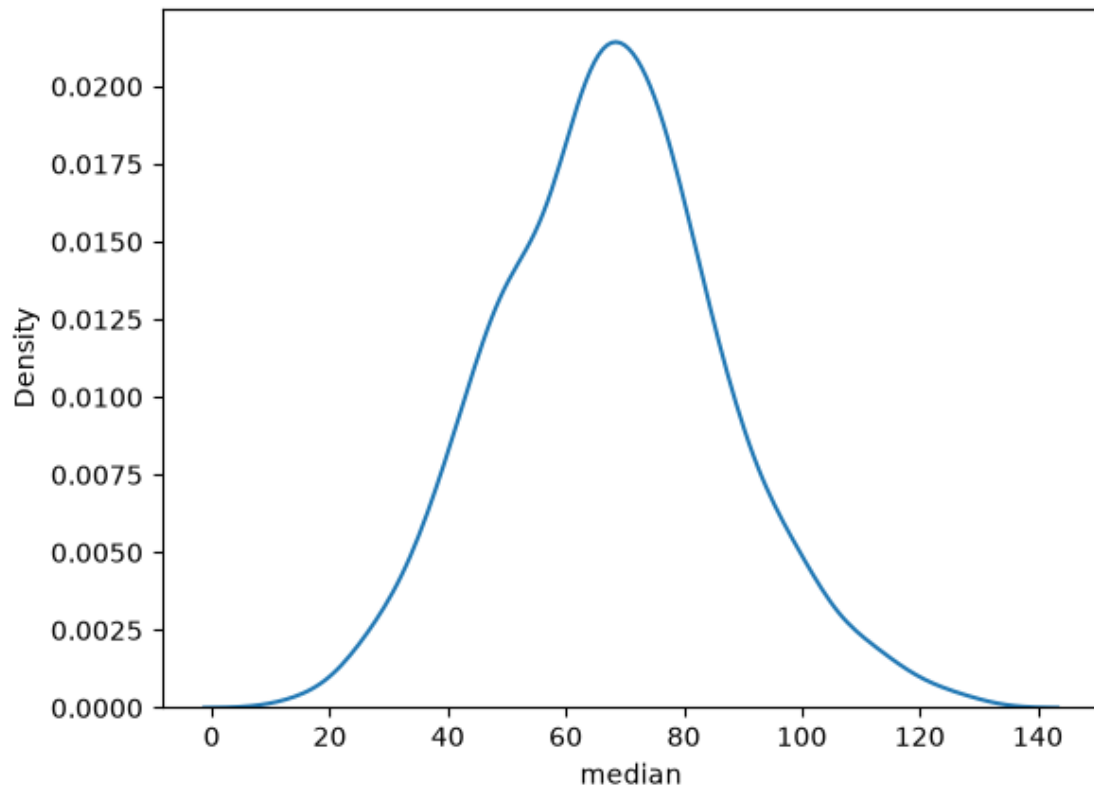


2. Now group by year-month and compute the **median** and **maximum** precipitation nationwide. Plot the ECDF and KDE of these two statistics.

```
[7]: grouped = df.groupby(['year', 'month'])['prcp_total_mm'].agg(['median', 'max'])
```

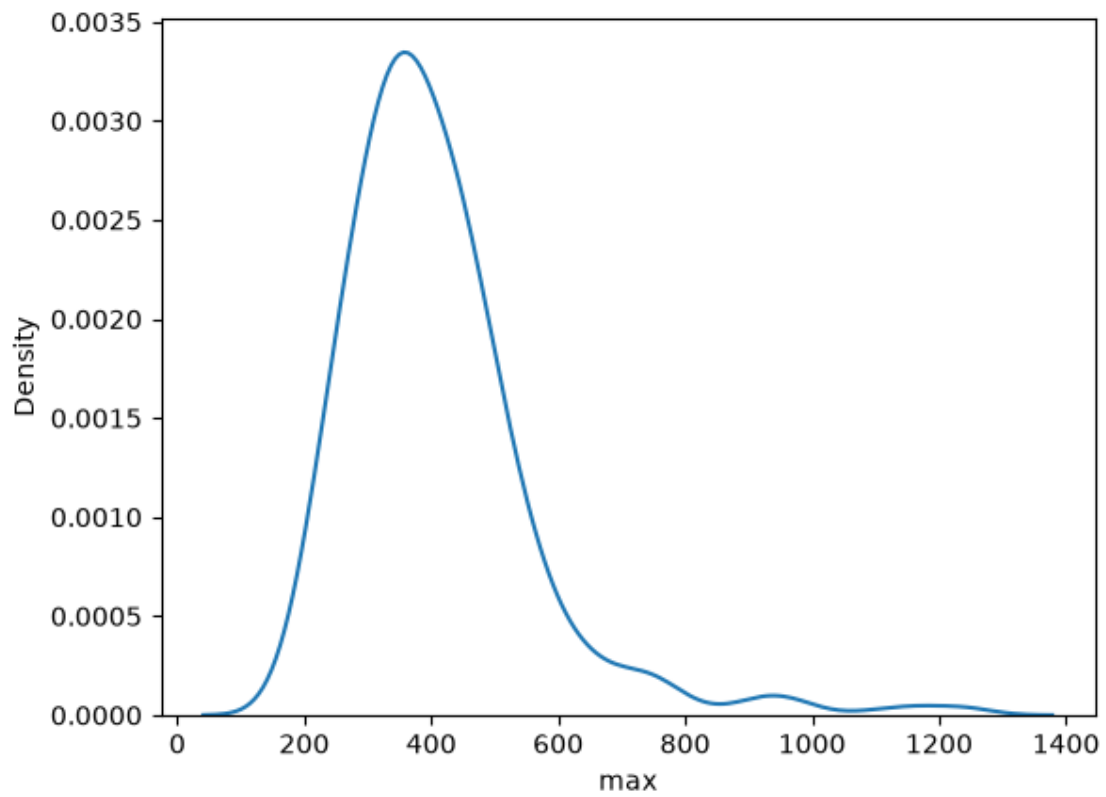
```
[8]: sns.kdeplot( data = grouped, x = 'median')
```

```
[8]: <Axes: xlabel='median', ylabel='Density'>
```



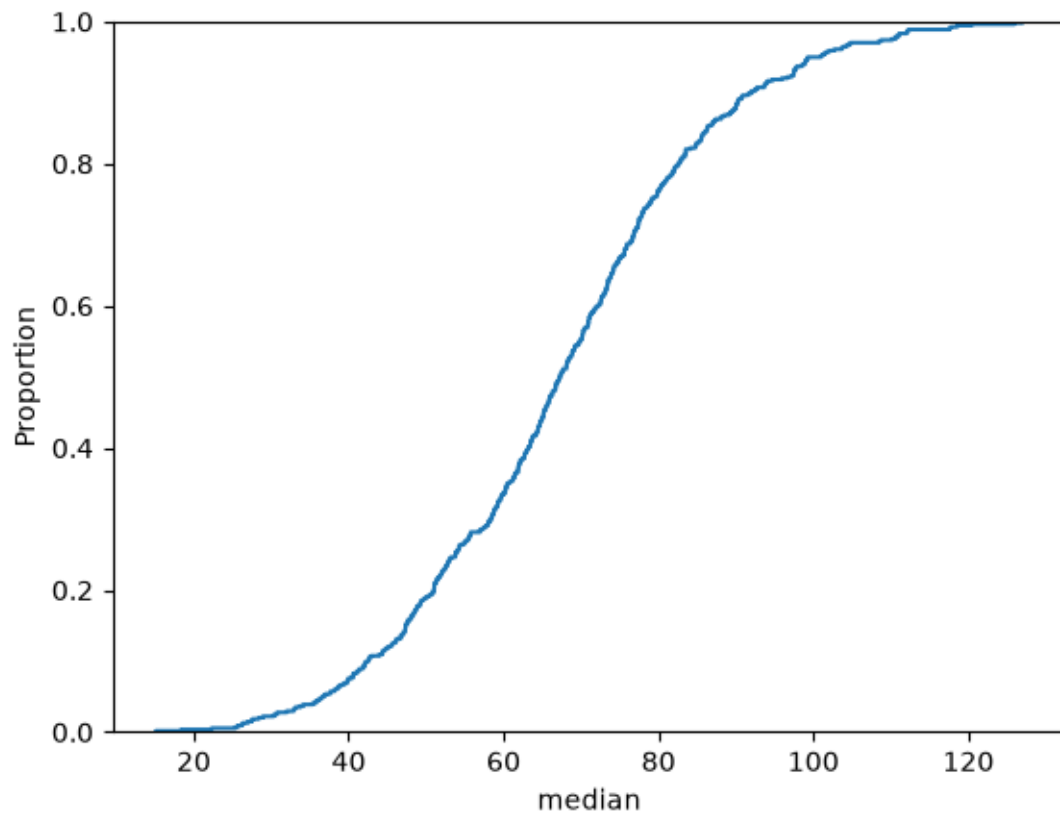
```
[9]: sns.kdeplot( data = grouped, x = 'max')
```

```
[9]: <Axes: xlabel='max', ylabel='Density'>
```



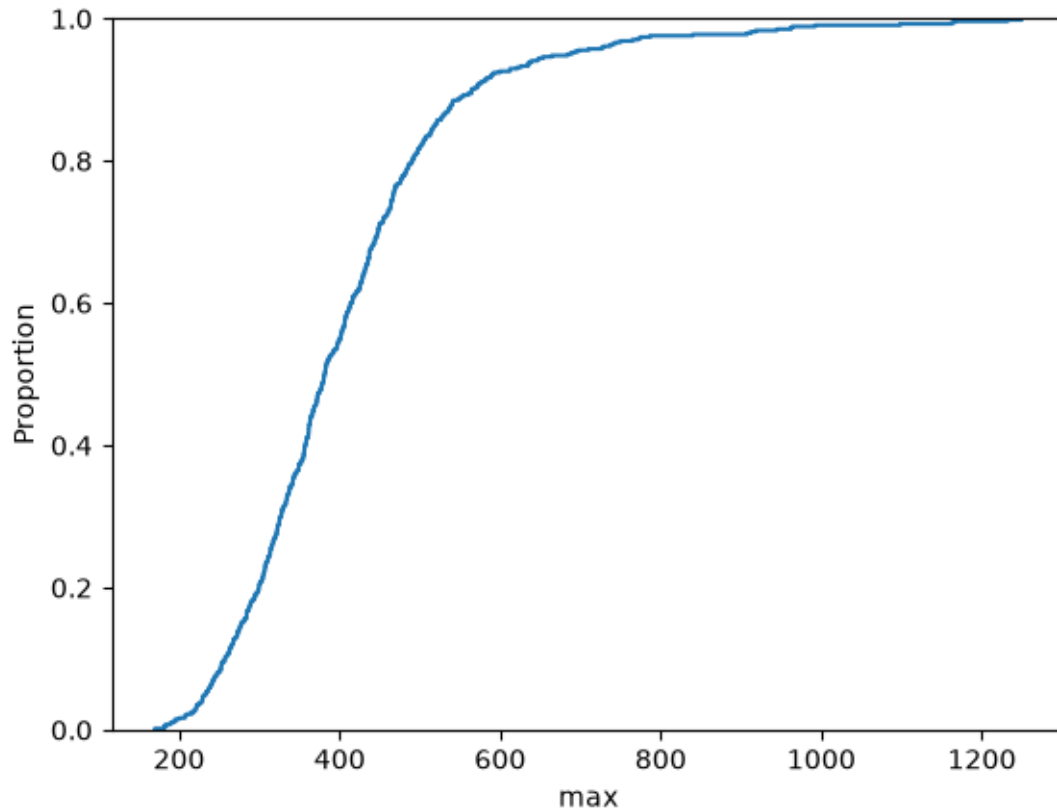
```
[10]: sns.ecdfplot( data = grouped, x = 'median')
```

```
[10]: <Axes: xlabel='median', ylabel='Proportion'>
```



```
[11]: sns.ecdfplot( data = grouped, x = 'max')
```

```
[11]: <Axes: xlabel='max', ylabel='Proportion'>
```



3. How do your results in part 2 compare to your results in part 1?

Compare and contrast the ECDFs and KDEs from parts 1 and 2. Explain why they are different:

The graphs from part 1 are different from the graphs from part 2, as they look to be a bit smoother. This is due to the fact that part 1 is displaying individual rainfall observations and part 2 is grouping the results by month, and then plotting the median and max of the data. The plots from part 1 show the rainfall measurements are mostly small, but with some high events. For the ECDF plot of all of the data, it looks like an exponential distribution, whereas the plots of the median and max appear more similar to a normal distribution. The KDE plot looks similar, but the median and max graphs have a less sharp peak. This is because any outliers have much less influence on the median, while the max plots specifically pick the highest rainfall values, resulting in a distribution shifted towards higher values.

4. Now define a state's rainfall differently: take the **median** of the 5 stations in each month, then find the **maximum** across the state's 12 months in a year — the “wettest month.” For each state, what's the expected value of the wettest month? Plot the ECDF of Virginia's annual maxima (the wettest month in each of the 45 years), and overlay the analytical prediction  $F_{\max}(x) = \hat{F}(x)^{12}$ .

```
[12]: # One rainfall value per state-month: median across stations
state_month = (
```

```

    df.groupby(['year', 'month', 'state'], as_index=False)['prcp_total_mm']
        .median()
)

# Wettest month of each year, per state: max over the 12 state-month values
state_year_max = (
    state_month.groupby(['year', 'state'], as_index=False)['prcp_total_mm']
        .max()
)

exp_wettest = state_year_max.groupby("state", as_index=False).
    ↪agg(E_wettest_month=("prcp_total_mm", "mean"))
exp_wettest.loc[exp_wettest["state"] == "VA"]

state = "VA"

levels = (
    state_month.loc[state_month["state"] == state, "prcp_total_mm"]
        .to_numpy()
)

observed_max = (
    state_year_max.loc[state_year_max["state"] == state, "prcp_total_mm"]
        .to_numpy()
)

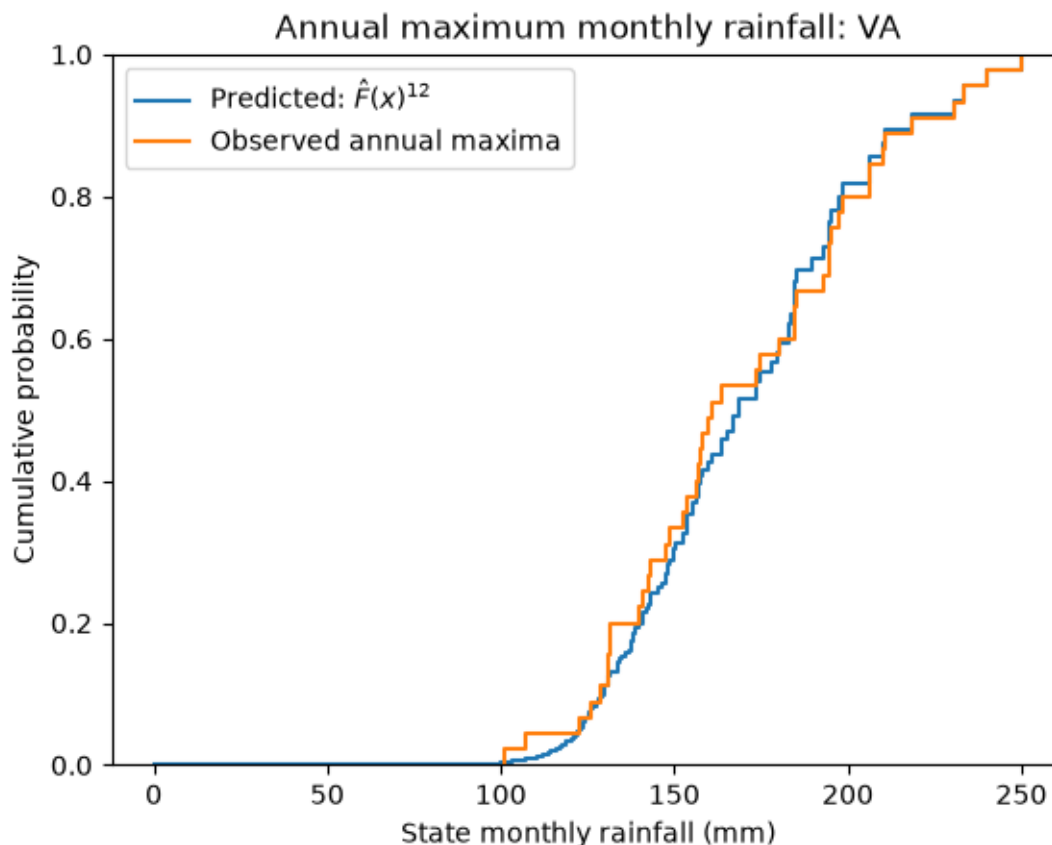
x = np.sort(np.unique(levels))
F_hat = np.searchsorted(np.sort(levels), x, side="right") / len(levels)

plt.step(
    x,
    F_hat**12,
    where="post",
    label=r"Predicted:  $\hat{F}(x)^{12}$ "
)

sns.ecdfplot(x=observed_max,
    label="Observed annual maxima")

plt.xlabel("State monthly rainfall (mm)")
plt.ylabel("Cumulative probability")
plt.title(f"Annual maximum monthly rainfall: {state}")
plt.legend()
plt.show()

```



How close is the fit here? If it differs from question 4's fit, why:

The fit here is pretty close, especially around the 200 mark for state monthly rainfall. Areas where the fit is not great are 100-125 and around 130-140. The wettest month is the maximum of the monthly values which is what is predicted. There are only 45 annual maxima. Both curves closely resemble the normal curve shown in the ECDF plot in part 2.

5. **A question for FEMA.** So far you've worked with *monthly* rainfall totals. Switch now to the daily station-level file (`ghcn_prcp_daily.parquet`) — reload `df` from it, and don't mix it with the monthly data above.

A day with more than 120mm of rain is a serious flood/landslide risk. On any single station-day, that's rare. But a state doesn't get one chance a month — it gets  $5 \text{ stations} \times (\text{days in the month})$  chances. FEMA doesn't care about the average day, or the single worst day on record; it cares about the probability that *some* station has a disaster day at some point during the month.

For each state-month, estimate

$$P(\text{at least one station-day exceeds 120mm}) = 1 - \hat{F}(120)^n$$

where  $\hat{F}$  is the empirical CDF of a single station-day's rainfall for that state and calendar month (pooled across all years), and  $n$  is the number of station-days in one occurrence of that

month (5 stations  $\times$  days in the month). Report the 20 highest-risk state-months.

```
[13]: # Fresh load -- this question uses only the daily file, not the monthly one
      ↪above.
      #df_daily = pd.read_parquet("data/ghcn_prdp_daily.parquet")

      url = (
          "https://www.dropbox.com/scl/fi/"
          "pjmyop3fqjdy8oabkc4ab/ghcn_prdp_daily.parquet"
          "?rlkey=otxenczb3pxgzz53kc1i5zxww&dl=1"
      )
      df_daily = pd.read_parquet(url)
      df_daily = df_daily.loc[df_daily["observed"]]

      threshold = 120 # mm in a single day

      def disaster_risk(group):
          F_hat = (group["prcp_mm"] <= threshold).mean()
          n = len(group) / group["year"].nunique() # station-days in one year's
          ↪occurrence of this month
          return 1 - F_hat**n

      risk = (
          df_daily.groupby(["state", "month"])
              .apply(disaster_risk, include_groups=False)
              .rename("p_any_station_day_exceeds_120mm")
              .reset_index()
              .sort_values("p_any_station_day_exceeds_120mm", ascending=False)
      )

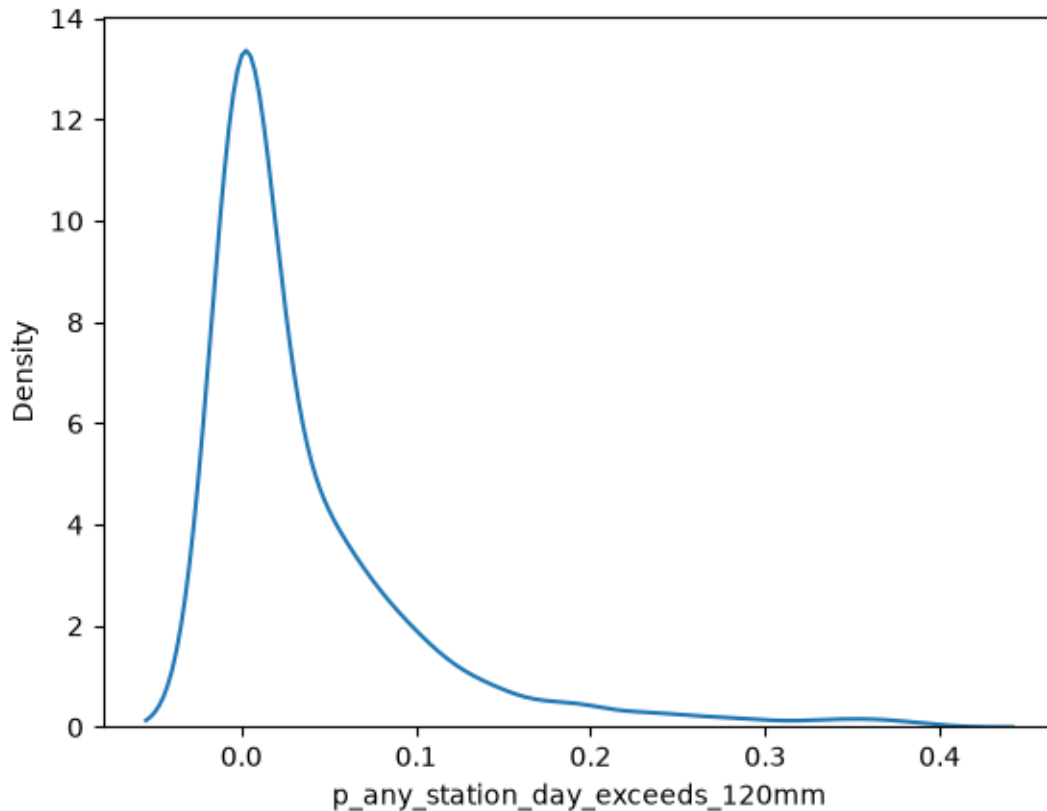
      sns.kdeplot(risk['p_any_station_day_exceeds_120mm'])

      risk.head(20)
```

```
[13]:
```

|     | state | month | p_any_station_day_exceeds_120mm |
|-----|-------|-------|---------------------------------|
| 122 | HI    | 3     | 0.387166                        |
| 104 | FL    | 9     | 0.373367                        |
| 130 | HI    | 11    | 0.359242                        |
| 208 | LA    | 5     | 0.359229                        |
| 131 | HI    | 12    | 0.344789                        |
| 121 | HI    | 2     | 0.330060                        |
| 105 | FL    | 10    | 0.330026                        |
| 476 | SC    | 9     | 0.299512                        |
| 207 | LA    | 4     | 0.299512                        |
| 319 | NC    | 8     | 0.283726                        |
| 536 | VA    | 9     | 0.267604                        |
| 101 | FL    | 6     | 0.267604                        |

|     |    |    |          |
|-----|----|----|----------|
| 211 | LA | 8  | 0.267597 |
| 209 | LA | 6  | 0.251113 |
| 212 | LA | 9  | 0.234253 |
| 127 | HI | 8  | 0.234248 |
| 215 | LA | 12 | 0.234248 |
| 117 | GA | 10 | 0.234248 |
| 103 | FL | 8  | 0.234248 |
| 120 | HI | 1  | 0.217012 |



Which states and months face the most risk of a greater-than-120mm day? How high is the probability? Among all state-months, describe the density of high precipitation days:

Hawaii in March, November, December and February, Florida in September and October, and Louisiana may have the highest probability for a greater than 120mm day. The probabilities are around 33-38%. Hawaii has six occurrences of risk for a greater-than-120mm day. Louisiana also has six occurrences, but with lower probabilities.

The density of high precipitation days appears to show its highest density between 0.0-0.1 when it precip may exceed 120mm. This means that it is not very likely across the 50 states and 12 months for a high precipitation day, however there are some greater values represented on the graph.

**Step Two.**

Our second data set is daily and monthly U.S. stock market returns. Instead of storms across space (five stations in a state) or across the calendar (twelve months in a year), the “sample” is the set of trading days within a month, and we ask about the best and worst day in that sample.

```
[15]: df = pd.read_parquet("ff_market_daily.parquet")

print(f"{len(df):,} trading days")
print(f"{df['date'].min().date()} through {df['date'].max().date()}")
df.head()
```

```
26,296 trading days
1926-07-01 through 2026-07-31
```

```
[15]:
```

|   | date       | year | month | month_start | trading_day_of_month | \ |
|---|------------|------|-------|-------------|----------------------|---|
| 0 | 1926-07-01 | 1926 | 7     | 1926-07-01  | 1                    |   |
| 1 | 1926-07-02 | 1926 | 7     | 1926-07-01  | 2                    |   |
| 2 | 1926-07-06 | 1926 | 7     | 1926-07-01  | 3                    |   |
| 3 | 1926-07-07 | 1926 | 7     | 1926-07-01  | 4                    |   |
| 4 | 1926-07-08 | 1926 | 7     | 1926-07-01  | 5                    |   |

|   | mkt_excess_return_pct | risk_free_return_pct | market_return_pct | \ |
|---|-----------------------|----------------------|-------------------|---|
| 0 | 0.09                  | 0.01                 | 0.10              |   |
| 1 | 0.45                  | 0.01                 | 0.46              |   |
| 2 | 0.17                  | 0.01                 | 0.18              |   |
| 3 | 0.09                  | 0.01                 | 0.10              |   |
| 4 | 0.22                  | 0.01                 | 0.23              |   |

|   | absolute_market_return_pct |
|---|----------------------------|
| 0 | 0.10                       |
| 1 | 0.46                       |
| 2 | 0.18                       |
| 3 | 0.10                       |
| 4 | 0.23                       |

1. Plot the KDE and ECDF of daily market returns.

```
[16]: fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

sns.kdeplot(
    data=df,
    x="market_return_pct",
    cut=0,
    ax=axes[0],
)
axes[0].axvline(0, color="black", linewidth=1, alpha=0.6)
axes[0].set(
    title="KDE of daily U.S. market returns",
    xlabel="Daily market return (%)",
    ylabel="Estimated density",
```

```

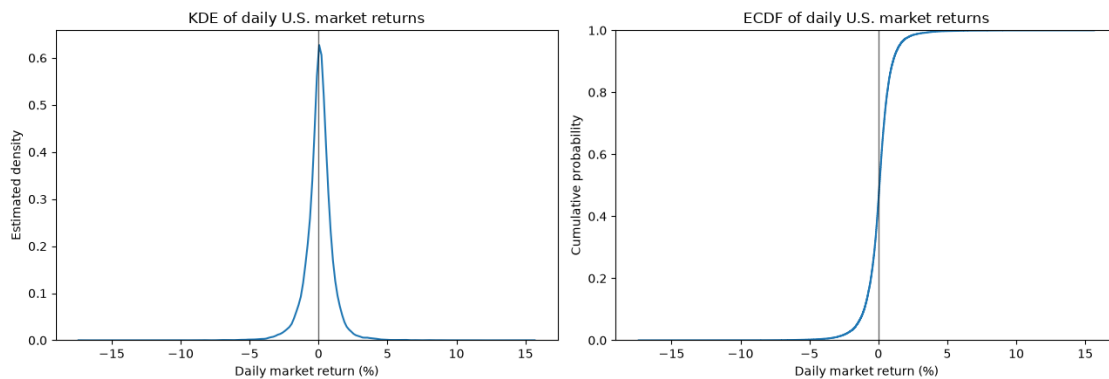
)

sns.ecdfplot(
    data=df,
    x="market_return_pct",
    ax=axes[1],
)

axes[1].axvline(0, color="black", linewidth=1, alpha=0.6)
axes[1].set(
    title="ECDF of daily U.S. market returns",
    xlabel="Daily market return (%)",
    ylabel="Cumulative probability",
)

plt.tight_layout()
plt.show()

```



2. For each calendar month, compute the **best** (max) and **worst** (min) daily return — the first and  $n$ -th order statistics of that month's trading days. Plot the KDE of the monthly best and worst returns, overlaid.

```

[17]: monthly = (
    df.groupby(["year", "month"], as_index=False)
        .agg(
            month_start=("month_start", "first"),
            trading_days=("date", "size"),
            worst_return_pct=("market_return_pct", "min"),
            best_return_pct=("market_return_pct", "max"),
            median_return_pct=("market_return_pct", "median"),
        )
)

# Put losses and gains on the same positive-magnitude scale.

```

```

monthly["worst_loss_pct"] = -monthly["worst_return_pct"]

print(f"{len(monthly):,} calendar months")
print(
    f"Trading days per month: {monthly['trading_days'].min()} "
    f"to {monthly['trading_days'].max()}"
)
monthly.head()

```

1,201 calendar months

Trading days per month: 15 to 27

```

[17]:   year  month month_start  trading_days  worst_return_pct  best_return_pct  \
0  1926     7  1926-07-01         25         -0.72         1.10
1  1926     8  1926-08-01         26         -1.39         0.85
2  1926     9  1926-09-01         24         -0.98         0.91
3  1926    10  1926-10-01         25         -1.81         1.53
4  1926    11  1926-11-01         24         -0.62         0.67

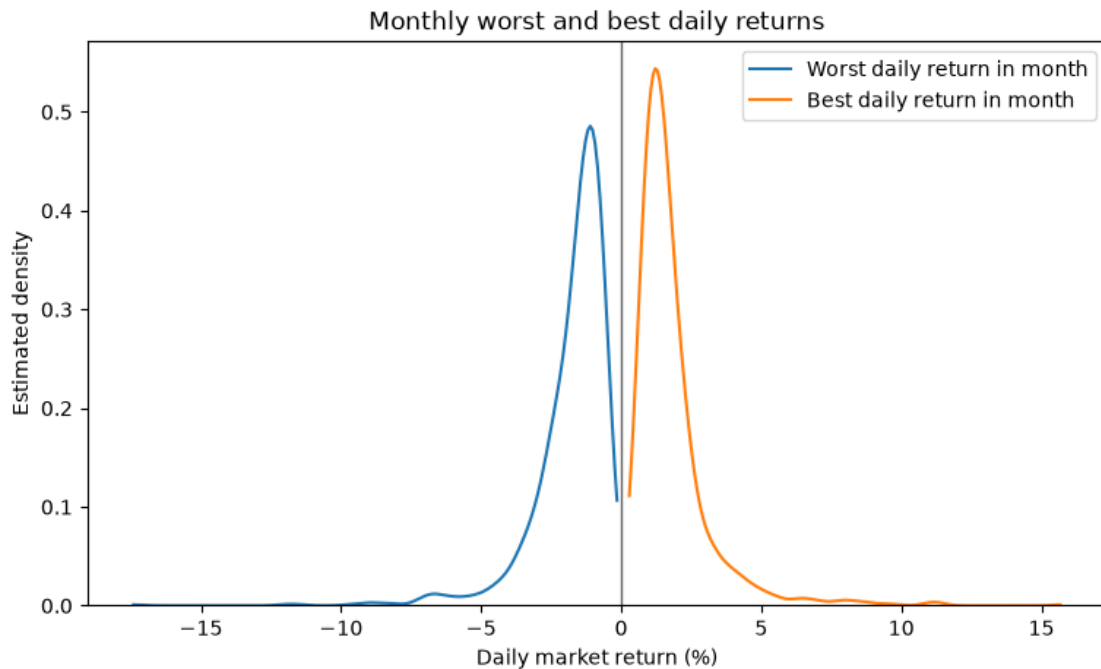
      median_return_pct  worst_loss_pct
0              0.18         0.72
1              0.32         1.39
2             -0.03         0.98
3             -0.09         1.81
4              0.19         0.62

```

```

[18]: plt.figure(figsize=(9, 5))
sns.kdeplot(
    x=monthly["worst_return_pct"],
    label="Worst daily return in month",
    cut=0,
)
sns.kdeplot(
    x=monthly["best_return_pct"],
    label="Best daily return in month",
    cut=0,
)
plt.axvline(0, color="black", linewidth=1, alpha=0.6)
plt.xlabel("Daily market return (%)")
plt.ylabel("Estimated density")
plt.title("Monthly worst and best daily returns")
plt.legend()
plt.show()

```



Describe the symmetry (or lack of it) between the best- and worst-day densities. Compare them to the overall populations of daily returns. Use code as appropriate to make these comparisons, and then write out your answer in a markdown cell.

```
[20]: summary = pd.DataFrame({
    "daily": df["market_return_pct"],
    "worst": monthly["worst_return_pct"],
    "best": monthly["best_return_pct"],
})
summary.describe()
```

```
[20]:
```

|       | daily        | worst       | best        |
|-------|--------------|-------------|-------------|
| count | 26296.000000 | 1201.000000 | 1201.000000 |
| mean  | 0.043005     | -1.777868   | 1.807460    |
| std   | 1.077241     | 1.397024    | 1.390837    |
| min   | -17.410000   | -17.410000  | 0.290000    |
| 25%   | -0.390000    | -2.210000   | 1.040000    |
| 50%   | 0.080000     | -1.410000   | 1.440000    |
| 75%   | 0.510000     | -0.940000   | 2.040000    |
| max   | 15.670000    | -0.150000   | 15.670000   |

```
[21]: # ai was used to generate this code

from scipy.stats import gaussian_kde
```

```
def kde_mode(series, n_points=2000):
    data = series.dropna().to_numpy()
    kde = gaussian_kde(data)
    xs = np.linspace(data.min(), data.max(), n_points)
    density = kde(xs)
    i = density.argmax()
    return xs[i], density[i]

worst_x, worst_y = kde_mode(monthly["worst_return_pct"])
best_x, best_y = kde_mode(monthly["best_return_pct"])

print(f"Worst-return mode: x = {worst_x:.3f}%, density = {worst_y:.4f}")
print(f"Best-return mode: x = {best_x:.3f}%, density = {best_y:.4f}")
```

Worst-return mode: x = -1.100%, density = 0.4856

Best-return mode: x = 1.236%, density = 0.5441

The best-day density has a higher, more concentrated peak than the worst-day density, and the worst-day tail is longer, with a noticeable bump around -6 to -7%. Both have a few smaller bumps in their tails, but overall the two plots are roughly mirror images of each other. Daily returns are centered near 0 with a standard deviation of about 1. The best and worst days have a standard deviation of about 1.4 and medians of about +/- 1.4.

Visually, the two sides of the graph look very similar, but computing the location of the maximums shows more of the difference between the two graphs. On the worst side, we see the maximum occurring at (-1.1%, .4856). The best side returns max out at (1.236%, 0.5441) This shows that the worse side returns are less drastic of a percentage than the better side. We can also see a second hump on the worst side, meaning that there are some days that are much worse than the average. We do not see a similar second hump on the best case side, which is further proved by the density being higher for the sole hump.

3. Losses and gains are on opposite sides of zero, which makes them hard to compare directly. Put them on the same scale by looking at the *magnitude* of the worst loss (`-worst_return_pct`) against the magnitude of the best gain (`best_return_pct`). Plot both the KDE and ECDF.

```
[22]: fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
```

```
sns.kdeplot(
    x=monthly["worst_loss_pct"],
    label="Worst loss magnitude",
    cut=0,
    ax=axes[0],
)
sns.kdeplot(
    x=monthly["best_return_pct"],
    label="Best gain",
    cut=0,
    ax=axes[0],
```

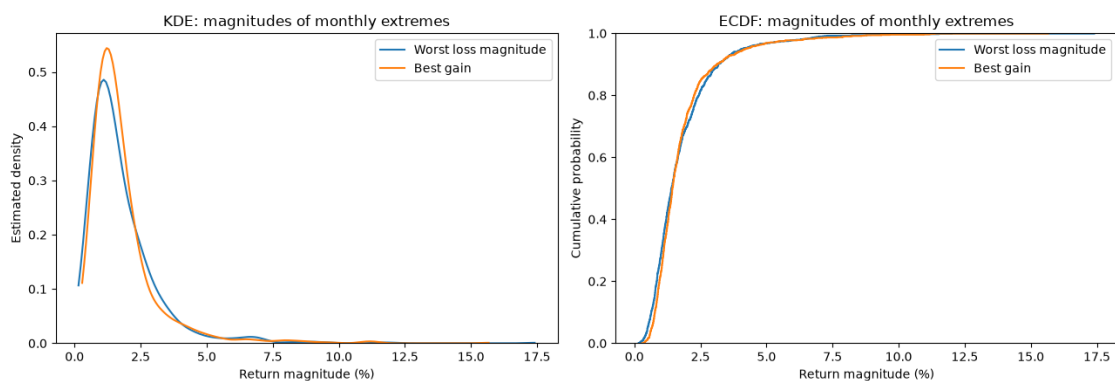
```

)
axes[0].set(
    title="KDE: magnitudes of monthly extremes",
    xlabel="Return magnitude (%)",
    ylabel="Estimated density",
)
axes[0].legend()

sns.ecdfplot(
    x=monthly["worst_loss_pct"],
    label="Worst loss magnitude",
    ax=axes[1],
)
sns.ecdfplot(
    x=monthly["best_return_pct"],
    label="Best gain",
    ax=axes[1],
)
axes[1].set(
    title="ECDF: magnitudes of monthly extremes",
    xlabel="Return magnitude (%)",
    ylabel="Cumulative probability",
)
axes[1].legend()

plt.tight_layout()
plt.show()

```



Are large losses more extreme than large gains, or about the same:

In the KDE plot, we can see that the curves have the same shape, but the best gain has a much higher peak than the worst loss magnitude. The worst loss magnitude distribution has a longer tail for both ECDF and KDE.

4. Compute the median, 75th, 90th, 95th, and 99th percentile of both magnitudes from question

3. In what fraction of months does the worst loss exceed the best gain in magnitude?

```
[23]: tail_summary = pd.DataFrame(
    {
        "Worst loss magnitude": monthly["worst_loss_pct"],
        "Best gain": monthly["best_return_pct"],
    }
).quantile([0.50, 0.75, 0.90, 0.95, 0.99]).T

tail_summary.columns = ["median", "75%", "90%", "95%", "99%"]

print(
    "Fraction of months in which the worst loss exceeded the best gain: "
    f"{(monthly['worst_loss_pct'] > monthly['best_return_pct']).mean():.1%}"
)

tail_summary.round(2)
```

Fraction of months in which the worst loss exceeded the best gain: 44.8%

```
[23]:
```

|                      | median | 75%  | 90%  | 95%  | 99%  |
|----------------------|--------|------|------|------|------|
| Worst loss magnitude | 1.41   | 2.21 | 3.18 | 4.06 | 6.97 |
| Best gain            | 1.44   | 2.04 | 3.12 | 4.21 | 7.93 |

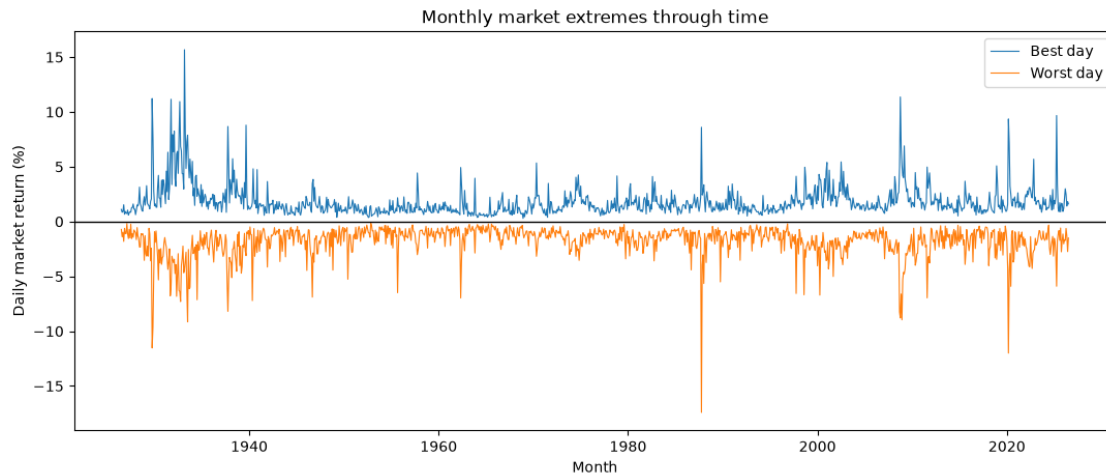
What does this tail comparison say about the risk of losses vs. gains:

The fraction of months where the worst loss exceeds the best gain is 44.8%, meaning that in most months, the best gain out performs the worst loss. The tail comparison shows that losses and gains are generally similar in magnitude, but extreme gains can be larger than extreme losses. However, losses are not consistently larger than gains. At the 99th percentile, the best gain, 7.93, is greater than the worst loss, 6.97, suggesting that the largest positive can exceed the largest negative value

5. Plot the monthly best and worst daily returns over time. Do the extremes look stable across this ~100-year history, or are there periods where they're much larger? Can you identify any famous market events?

```
[24]: plt.figure(figsize=(13, 5))
plt.plot(
    monthly["month_start"],
    monthly["best_return_pct"],
    linewidth=0.8,
    label="Best day",
)
plt.plot(
    monthly["month_start"],
    monthly["worst_return_pct"],
    linewidth=0.8,
    label="Worst day",
)
plt.axhline(0, color="black", linewidth=1)
plt.xlabel("Month")
```

```
plt.ylabel("Daily market return (%)")
plt.title("Monthly market extremes through time")
plt.legend()
plt.show()
```



Do upside and downside risk tend to go together (volatility), or does the market exhibit distinct periods in which the good shocks are concentrated and the bad shocks are concentrated? What implication does this have for investing?:

The market upside and downside seem to go together, meaning that they are volatile. Their spikes closely mirror each other, except for in 1990, when the negative is much farther out than the positive. This means that even if there is potential in a large gain it almost equally comes with great risk. Some months have unusually large gains or losses without an equal movement in the opposite direction. We can see the Great Depression, 1990 recession and 2008 financial crisis very clearly on the graph, as they are large spikes.

## 1.7 Wrap-up

This week has focused on kernel density estimation, particularly estimating the PDF from data. This lab takes that a step further: We can estimate and quantify the densities and distributions of random variables that build off of observed phenomena. What is the difference between the ECDF and KDE? Explain how these estimations today give us different information than an ECDF or KDE plot of the underlying data, but quantify information about the behavior of a different random variable.

KDE tells us how concentrated are the scores around a value, while ECDF tells us what proportion of scores are at or below a value. The estimations today give us different information than an ECDF or KDE plot of underlying data because we are focusing on a statistic, either median or maximum, or extreme value compared to an average dataset value. The ECDF estimates the CDF for a sample. The KDE is a smooth curve that estimates the PDF for a sample. This allows the data analysis to not focus on the overall average, but instead allow focus and analysis of a specific statistic or variable. Both of these help to get insight on the magnitude/frequency of the gains and

losses.